Ders 13 Genişletilmiş Notu: Dönem Projesi Sunumları

Ders 13 Genişletilmiş Notu: Dönem Projesi Sunumları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Sunumların Düzeni
- 2. Laptop Fiyat Karşılaştırma Tarayıcısı (Scrapy ve Django)
- 3. Kredi Kartı Sahtecilik Tespiti
- 4. Twitter Haber Başlığı Sınıflandırma
- 5. MovieLens ile Film Önerisi (Pearson Korelasyonu)
- 6. Ürün Takip ve İsim Tabanlı Ürün Önerisi
- 7. Banka Müşteri Kaybı (Churn) Tahmini
- 8. iPhone Fiyat Karşılaştırma Tarayıcısı
- 9. TEDx Video Öneri Sistemi
- 10. Türkçe Haber Metni Kategorilendirme ve Bulutta Yayınlama
- 11. Operatör Yorumlarında Kümeleme
- 12. Tweet Duygu Analizi (Logistic Regression)
- 13. Kullanıcı ve Öğe Tabanlı Film Önerisi (k-NN)
- 14. Reddit Yorum Kaldırma Tahmini
- 15. E-posta Spam Tespiti
- 16. Kitap Fiyat Karşılaştırma Tarayıcısı
- 17. Akademik Makale Başlığı Üretme (LSTM ve T5)
- 18. Web Sunucusu Log Analizi ve Birliktelik Kuralları
- 19. Gazete Haberlerinde Taraf Belirleme
- 20. Ayakkabı Fiyat Tarayıcısı ve Filtreleme Eksikliği
- 21. Şarkı Öneri Sistemi ve Değerlendirme Eksikliği
- 22. Hazır Kütüphaneyle Tweet Duygu Analizi
- 23. Web Sayfasından Anahtar Kelime Çıkarma
- 24. Telefon Yorumları Tarayıcısı (Scrapy, MongoDB, React)
- 25. LG Televizyon Fiyat Tarayıcısı (Scrapy ve Flask)
- 26. Restoran Yorumlarında Duygu Analizi
- 27. Emoji Tabanlı Duygu Kümeleme
- Örnek Sorular
- Dersin Toparlaması

1. Sunumların Düzeni

Dönemin son dersinde konu anlatılmadı. Bütün kayıt dönem projesi sunumlarından oluşuyor. Hoca proje gruplarını listeledi ve grupları Zoom'un bekleme odasına (waiting room) aldı. Her grubu sırayla toplantıya alıp projesini anlattırdı.

Hoca her gruptan aynı akışı istedi:

- Önce öneri (proposal) belgesi üzerinden projede ne yapılmak istendiğini hatırlatmak.
- Sonra çalışan bir demo göstermek.
- En sonda editörü açıp demonun arkasındaki kodu anlatmak.

Ekran paylaşımı yapabilmek grubun sorumluluğuydu. Hoca sunumlarda öneri belgesine uyumu, derste işlenen tekniklerin kodun içinde görülmesini ve sonuçların değerlendirilmesini sorguladı. Projeler dersin dört ana başlığına dağılıyordu: web tarayıcı (crawler) uygulamaları, metin sınıflandırma, öneri sistemleri (recommendation systems) ve fikir madenciliği (opinion mining) ile duygu analizi (sentiment analysis).

2. Laptop Fiyat Karşılaştırma Tarayıcısı (Scrapy ve Django)

İlk sunulan proje bir web tarayıcı uygulamasıydı. Hoca konuyu şöyle özetledi: Hepsiburada ve n11 sitelerinden laptop verisi çekilecek, sonuç hibrit filtreleme yapılabilen bir sayfada gösterilecek. Grup fikrin, uzaktan eğitim döneminde çevrelerinden sık gelen "hangi bilgisayarı alalım" sorusundan çıktığını anlattı. Amaç marka, fiyat, özellikler ve ürünün satıldığı sayfanın bağlantısını toplayıp karşılaştırmayı kolaylaştırmaktı.

2.1 Selenium'dan Scrapy'ye Geçiş

Grup önce Selenium'u denedi. Selenium tarayıcıyı sürekli çalıştırdığı için yavaştı. Araştırma sonunda Scrapy'yi buldular ve çok daha hızlı ilerlediğini gördüler.

Scrapy projesinin hazır dosyalarına (settings, pipelines, middlewares) dokunmadılar. Yalnız örümcek (spider) kısmını yazdılar. Hepsiburada ve n11 için iki ayrı örümcek sınıfı vardı. Her örümcekte şunlar tanımlıydı:

- Örümceğin adı (`name`) ve başlangıç URL'si.
- Sayfalar arasında ilerlemek için bir sayfa numarası değişkeni.
- Dosyaya yazarken kullanılan bir kontrol sayacı.
- Sonuçları tutmak için `items` içinde fiyat, bağlantı ve özellik alanları.

Fiyat, bağlantı ve özellik alanları `response.xpath(...)` ile HTML içindeki ilgili yol izlenerek çekildi. Sonuçlar `items` nesnesine atandı.

2.2 CSV'ye Yazma ve Veri Temizleme

Çekilen her sayfa bir CSV dosyasına yazıldı. Alan adları marka, fiyat, özellik ve bağlantıydı. Bir sayfadaki bilgisayar sayısı kadar dönen bir döngüyle satırlar yazıldı. Özellik metninin içinden yalnız marka (Lenovo, Asus, Casper gibi) ayrı bir değişkene alındı.

İki pratik sorun çıktı:

- **Fiyatın metin olması:** Fiyat aralığına göre süzme yapabilmek için fiyatın sayı olması gerekiyordu. Virgül noktaya çevrildi ve fiyat CSV'ye ondalık sayı (float) olarak yazıldı. Bu yapılmadan önce boş fiyat aralığı girildiğinde arayüz hata veriyordu. n11'de fiyat farklı biçimde geldiği için ek değiştirme işlemleri gerekti.
- **Alan adlarının veri gibi görünmesi:** Alan adları CSV'nin ilk satırına yazıldığı için web sayfasında veri satırı gibi çıkıyor ve süzmeyi bozuyordu. İlk satır silindi.

Sayfalar arasında ilerlemek için bir sonraki sayfanın URL'si `format` fonksiyonuyla sayfa numarası değiştirilerek üretildi. 50 sayfa olduğu için sayfa numarası 51'den küçük olduğu sürece numara artırılıp `parse` yeniden çağrıldı. Tarama `scrapy crawl n11` ve `scrapy crawl hepsiburada` komutlarıyla başlatılıyordu.

2.3 Django Arayüzü

Django tarafında `views` dosyası CSV dosyalarını açıp satırları dizilere aldı ve şablona gönderdi. Şablonda bir veri tablosu (data table) kütüphanesi yüklüydü. `for` döngüsüyle her ürünün markası, fiyatı, modeli ve bağlantısı tabloya basıldı. Demoda şunlar gösterildi:

- En düşük ve en yüksek fiyat girerek süzme. Örneğin üst sınır 4000 girildiğinde yalnız 4000 liranın altındaki bilgisayarlar geldi.
- Arama kutusuyla marka ya da özellik üzerinden süzme. Örneğin "Asus" ile birlikte fiyat sınırı ya da "4 GB" araması.
- Satırdaki bağlantıya tıklayınca ürünün satıldığı siteye gitme.

3. Kredi Kartı Sahtecilik Tespiti

İkinci grup, bitirme projesiyle bağlantılı bir sınıflandırma projesi sundu. Kredi kartı işlem verilerinde sahtecilik (fraud) aranıyordu. Grup, bitirme projesinde kullandıkları MOA aracına tam hâkim olmadıkları için aynı işi Python'da da yapıp sonuçları karşılaştırmak istediklerini söyledi. Sonuçlar MOA ile tutarlı çıkmıştı.

Dört model kullanıldı: Logistic Regression, Naive Bayes, Decision Tree ve SGD. Arayüz önce veri dosyasının adını, sonra eğitim yüzdesini istiyordu. Logistic Regression ile doğruluk yaklaşık %99 çıktı. Grup, sahtecilik sınıfının ne kadarının yakalandığını gösteren ek bir sütun da hazırlamıştı. Örneğin test verisinde 10 sahte işlem varsa 7'sinin bulunması %70 demekti. Ayrıca tek bir kaydı vektör olarak verip son seçilen sınıflandırıcıyla tahmin yaptırmak da mümkündü. Veri seti yaklaşık 140 MB olduğu için Google Drive bağlantısıyla gönderilmişti.

> **Ek bilgi (derste anlatılmadı):** Sahtecilik verileri genellikle çok dengesizdir: sahte işlemler toplamın çok küçük bir kısmıdır. Bu yüzden %99 gibi bir doğruluk tek başına başarı göstermez; her şeye "normal" diyen bir model de büyük olasılıkla benzer doğruluk alır. Grubun ayrıca gösterdiği "sahte işlemlerin yüzde kaçı bulundu" ölçüsü, sahtecilik sınıfının duyarlılığıdır (recall) ve bu tür problemlerde asıl bakılması gereken ölçülerden biridir.

4. Twitter Haber Başlığı Sınıflandırma

Üçüncü grup, Twitter'daki haber hesaplarının paylaştığı başlıkları sınıflandırdı. İngilizce çalıştılar. Bir haber veri setinde başlıklar ve kategoriler vardı ve beş kategori seçtiler. Tweetler Tweepy kütüphanesiyle çekildi.

Grup başta Google'ın yaklaşık 1,5 GB'lık önceden eğitilmiş Word2Vec vektör tablosunu kullanmayı denemişti. Hocanın önerdiği tekniklere geçince işin daha kolay olduğunu gördüler.

4.1 Arayüz ve Sonuçlar

Arayüz Python'un tkinter kütüphanesiyle yazılmıştı. Açılır menüden üç algoritmadan biri seçiliyordu. Seçilen algoritma eğitim verisiyle eğitiliyor, test verisi üzerinde sınıflandırma raporu (classification report) çıkarıyordu. Raporda her kategori için precision, recall, F1 ve örnek sayısı, en altta da doğruluk vardı. Doğruluklar şöyleydi:

Algoritma	Doğruluk
Decision Tree	%58
Random Forest	%67
Naive Bayes	%73

Arayüzün alt kısmında New York Times ve CNN hesaplarından çekilen son 30 tweet (her hesaptan 15) ve her biri için tahmin edilen kategori görünüyordu. Grup, istenen bir Twitter hesabının girilmesini de düşünmüş, ama hatalı girişte programın çökmesini engellemek zor olduğu için vazgeçmişti.

4.2 Veri Setinin Eskimesi

Canlı tweetlerde tahminler pek doğru değildi. Grup bunun nedenini açıkladı: Veri seti 2017'ye kadar yayımlanmış makalelerin başlıklarından oluşuyordu. Bu yüzden:

- Son tweetlerin çoğu COVID ile ilgiliydi. Veri setinde COVID örneği olmadığı için model bunları "Science and Health" gibi uygun kategoriye koyamıyordu.
- Veri seti Donald Trump'ın başkan olmadığı döneme aitti. O dönemde daha çok bir magazin figürü olduğu için, onunla ilgili tweetler "Politics" yerine "Culture" gibi kategorilere düşebiliyordu.

> **Ek bilgi (derste anlatılmadı):** Eğitim verisi ile modelin kullanıldığı dönemin verisi arasındaki dağılım farkına literatürde kavram kayması (concept drift) ya da veri

kayması (data drift) denir. Haber gibi hızla değişen alanlarda modelin güncel etiketli veriyle düzenli olarak yeniden eğitilmesi bu sorunu azaltır.

4.3 Kod

Kodda scikit-learn'ün Decision Tree, Random Forest ve Naive Bayes sınıflandırıcıları ile metni sayılara çeviren bir vektörleştirici (vectorizer) kullanıldı. Akış şöyleydi:

- Geliştirici hesabından alınan anahtarlarla Twitter API'sine bağlanıp iki hesabın tweetlerini çekmek.
- Tweetlerin sonundaki görsel bağlantılarını (`http` ile başlayan) düzenli ifadeyle (regex) temizlemek. Bu yapılmadığında sonuçlar anlamsız çıkıyordu.
- Veri setinden beş kategoriyi ve başlıkları alıp eşleştirmek, eğitim ve test olarak bölmek.
- Eğitim, test ve çekilen tweetleri ayrı ayrı vektörleştirmek. Vektörleştirici her kelimeye bir sayı verip her başlık için bir vektör oluşturuyordu.
- Kategorileri kodlayıcıyla (encoder) 0, 1, 2, 3, 4 sayılarına çevirmek.
- Eğitim verisinin sınıflar arasında eşit dağılması için örnekleme (sampling) yapmak.

Grup ilk başta yaklaşık 211 MB'lık veri setinin tamamıyla çalışırken özellik azaltma (feature reduction) yararlı olmuştu. Beş kategoriye inince özellik azaltma doğruluğu düşürdüğü için kaldırıldı. Veri seti data.world sitesinden alınmıştı ve erişim için kayıt gerekiyordu.

5. MovieLens ile Film Önerisi (Pearson Korelasyonu)

Dördüncü grup bir film öneri sistemi yaptı. MovieLens veri setini kullandılar. Grubun verdiği bilgiye göre veri setinde 20 milyondan fazla puan ve yaklaşık 140 bin kullanıcı vardı.

Yöntem şöyleydi:

1. En çok puan veren 100 kullanıcı bir alt küme olarak ayrıldı.
2. Sistemin kullanıcısından alınan puanlarla bu 100 kullanıcının puanları arasındaki Pearson korelasyonu hesaplandı.
3. En yüksek korelasyonlu (en benzer) 50 kullanıcı seçildi.
4. Her film için bu kullanıcıların puanı benzerlik değeriyle (similarity index) çarpılıp ağırlıklı puan bulundu.
5. Ağırlıklı puanların toplamı benzerliklerin toplamına bölünerek her film için ortalama bir öneri skoru elde edildi.
6. Skoru en yüksek 10 film önerildi.

Grubun verdiği örnek: Bir filme bir kullanıcı 4 vermiş ve benzerliği 0,7, başka bir kullanıcı 3,5 vermiş ve benzerliği 0,9 olsun. Öneri skoru şöyle bulunur (ara adım eklendi):

$$\text{skor} = \frac{4 \cdot 0{,}7 + 3{,}5 \cdot 0{,}9}{0{,}7 + 0{,}9} = \frac{2{,}8 + 3{,}15}{1{,}6} = \frac{5{,}95}{1{,}6} \approx 3{,}72$$

Hoca "item-based collaborative filtering mi kullandınız?" diye sordu. Grup "content-based yaptık" dedi ve bir kütüphane kullanmadan kendileri yazdıklarını söyledi. Önerilen filmler arayüzde değil, konsolda gösterilebildi.

> **Ders sonrası not:** Grup yöntemi "content-based" diye adlandırdı. Anlatılan yöntem (sistemin kullanıcısına en benzer kullanıcıları Pearson korelasyonuyla bulup onların puanlarının benzerlikle ağırlıklı ortalamasını almak) kullanıcı tabanlı işbirlikçi filtrelemedir (user-based collaborative filtering). İçerik tabanlı (content-based) yöntem kullanıcılara değil, filmlerin kendi özelliklerine (tür, oyuncu, etiket gibi) bakar.

6. Ürün Takip ve İsim Tabanlı Ürün Önerisi

Beşinci grubun amacı, farklı e-ticaret sitelerindeki ürünleri tek bir siteden takip etmek ve benzer ürün önermekti. Öneri belgesinde belli gruptaki ürünleri çekip kullanıcıya öneri yapmak ve fiyat değişimlerini göstermek yazıyordu.

6.1 Uygulamanın Akışı

- **Giriş:** Kullanıcı Google hesabıyla giriş yapıyordu. Her Google hesabı için Firebase'de bir tablo dinamik olarak oluşturuluyor, kullanıcının ürünleri orada tutuluyordu.
- **Ürün ekleme:** Kullanıcı ürünün bağlantısını yapıştırıyordu. Sistem arka planda ürünün adını, güncel fiyatını ve fotoğrafını tarayıp çekiyordu. Kullanıcı bir hedef fiyat (target price) ve not girebiliyordu.
- **Fiyat kontrolü:** Amaç, güncel fiyat hedef fiyatın altına düşünce kullanıcıya e-posta ya da bildirim göndermekti. "Check price" düğmesi vardı ama bu özellik tamamlanamamıştı. Örnekte bir ayakkabının fiyatı 330 TL'ydi ve kullanıcı 250 lira olunca haber almak istiyordu.
- **Filtreler:** Ürün adına göre arama, tarih aralığına göre süzme ve güncel fiyata göre sıralama vardı.

6.2 Öneri Yöntemi

"Start Crawler" düğmesi birkaç sitenin fırsat ürünlerini tarıyordu: n11'de indirimli ve yıldızlı ürünler, GittiGidiyor'da fırsat ürünleri, Trendyol'da çok satanlar kategorisi. Demoda bu veri setinde yaklaşık 500 ürün vardı. Amaç, kullanıcı bir ürün eklediğinde ona benzeyen ve indirimde olan bir ürünü sunmaktı.

Öneri yalnız ürün adlarına dayanıyordu, yani isim tabanlıydı:

1. Ürün adları TF-IDF vektörlerine çevrildi.
2. Eklenen ürünün vektörü ile diğer ürünlerin vektörleri arasındaki kosinüs benzerliği (cosine similarity) hesaplandı.
3. Kendisi dışındaki en benzer ürün kullanıcıya önerildi.

Demoda bir Bluetooth kulaklığa isim benzerliğinden dolayı başka bir Bluetooth cihazı, bir bebek arabasına başka bir bebek arabası önerildi. Hoca projeyi beğendiğini söyledi.

7. Banka Müşteri Kaybı (Churn) Tahmini

Altıncı sunum tek kişilik bir sınıflandırma projesiydi. Kaggle'dan bir bankanın müşteri verisi alınmıştı. Veride müşterilerin demografik bilgileri ve bankayla ilgili bilgileri (kredi kartı olup olmadığı gibi) vardı. Yaklaşık 10 bin örnek vardı. Amaç müşterinin bankadan ayrılıp ayrılmayacağını (churn) tahmin etmekti.

Kullanılan sınıflandırıcılar Decision Tree, Random Forest, Naive Bayes ve k-NN'di. SVM de eklenmişti ama yaklaşık 15 dakika bekleyip sonuç alamayınca denenmedi. En iyi sonucu Random Forest verdi.

Arayüzde şunlar seçilebiliyordu: CSV dosyası, algoritma ve parametreleri, kaydırma çubuğuyla eğitim-test oranı ve iki normalizasyon yöntemi (standart normalizasyon ve min-max normalizasyonu). Özellik seçimi (feature selection) denenmiş, ama özellik sayısı az ve korelasyonlar düşük olduğu için hiçbir özellik elenmemişti.

Kodda yapılan ön işleme şöyleydi:

- Bütün örneklerde tekil değer taşıyan üç sütun (satır numarası, müşteri kimliği ve benzeri) sınıfa bir bilgi katmadığı için atıldı.
- Kategorik veriler kodlandı (encoding). Kodlanmış kategorik sütunlar normalizasyona sokulmadı, çünkü normalize edilince garip sonuçlar çıkıyordu.
- Hedef sütun ayrıldı, seçildiyse normalizasyon uygulandı ve seçilen algoritma kullanıcının girdiği parametrelerle oluşturuldu.
- Karışıklık matrisi (confusion matrix) çizildi. Doğruluk elle, diğer üç ölçü scikit-learn'le hesaplandı. Precision, recall ve F-ölçüsü sınıf dengesizliğine bakmayan makro ortalama olarak gösterildi.

Öğrenci sınıf dengesizliğinin etkisini de açıkladı: Ayrılmayan müşteriler (non-churn) çok baskın olduğu için doğruluk %80'in üzerine çıkıyordu. Oysa ayrılan müşterilerin ancak yaklaşık yarısı doğru bulunuyordu. Tek bir müşteri için tahmin yapma özelliğini ekleyememişti.

8. iPhone Fiyat Karşılaştırma Tarayıcısı

Yedinci grup Python'da Scrapy ile dört siteden (Evkur, Vatan Bilgisayar, n11 ve bir site daha) iPhone ürünlerini listeledi. Her sitede şunlar çekildi:

- Ürünün adı (CSS seçicisiyle) ve fiyatı.
- `response.xpath` ile ürün görseli.
- Ayrıntı sayfasına gitmek için yönlendirme bağlantısı.

Veriler hem CSV'ye hem JSON'a yazıldı. Web sayfasında JSON dosyası kullanıldı. Arayüz ücretsiz bir barındırma (host) hizmetinde yayınlandı. Sayfada her ürün bir kart yapısıyla gösteriliyordu: görsel, ad, fiyat ve ayrıntı bağlantısı.

Demoda iki süzme gösterildi. Birincisi en yüksek fiyata göre: üst sınır düşürülünce 13 bin liralık ürünler listeden çıkıyordu. İkincisi siteye göre: yalnız Vatan Bilgisayar seçilince üç telefon kaldı, n11'de iPhone XR olmadığı için oradan sonuç gelmedi. Grup readme dosyasında URL'yi ve yerelde nasıl çalıştırılacağını anlatmıştı.

9. TEDx Video Öneri Sistemi

Sekizinci grup TEDx sitesindeki videolar için bir öneri sistemi geliştirdi. Önce videolar taranıyor, sonra toplanan veri üzerinde TF-IDF ile öneri yapılıyordu.

9.1 Tarayıcı

Tarayıcı önce video listesi sayfalarında her videonun başlığını ve URL'sini kaydediyor, sonra her URL'ye gidip ilgili etiketleri (related tags) ve video açıklamasını alıyordu. Sonuç URL, başlık, etiketler ve açıklama sütunlarıyla CSV'ye yazılıyordu. Grubun verdiği bilgiye göre listede yaklaşık 140 sayfa ve yaklaşık 4009 video vardı. Tam tarama 5 saati geçtiği için demoda yalnız ilk sayfa tarandı ve 36 video toplandı.

9.2 Öneri

Hoca hangi algoritmanın kullanıldığını sordu. Grup, başlıklar ve açıklama metinleri üzerinden bir TF-IDF matrisi oluşturduklarını, sonra benzerlik matrisine geçip benzerlikleri sıralayarak öneri yaptıklarını anlattı. Hoca bunu şöyle özetledi: Videonun etiketlerine ve başlığına bakarak bir vektör oluşturuyorsunuz, bu vektöre en yakın video vektörlerini bulup onları öneriyorsunuz. Grup kullanıcı bilgisi kullanmadıklarını, yalnız öğe (item) benzerliğine baktıklarını doğruladı.

Öneri başlığa, etiketlere ve açıklamaya göre ayrı ayrı yapılabiliyordu. Ayrıca üçü belirli ağırlıklarla birleştirilerek ortak bir öneri üretilebiliyordu. Kullanıcı başlığa, etiketlere ve açıklamaya ne kadar önem vereceğini kendisi belirliyordu. Arayüz yoktu, sistem konsolda çalışıyordu. Hocanın sorusu üzerine grup arayüze zaman kalmadığını söyledi.

10. Türkçe Haber Metni Kategorilendirme ve Bulutta Yayınlama

Dokuzuncu grup Türkçe metin kategorilendirme (text categorization) yaptı. Veri seti olarak Yıldız Teknik Üniversitesi Kemik doğal dil işleme grubunun yayımladığı bir haber veri seti kullanıldı. Haberler siyaset, ekonomi, spor gibi yedi kategoriye ayrılıyordu.

10.1 Modeller ve Arayüz

Flask ile bir web arayüzü yazıldı. İki özellik çıkarma yöntemi vardı: basit kelime torbası (bag of words, BoW) ve TF-IDF ağırlıklı kelime torbası. Her biriyle birkaç makine öğrenmesi modeli eğitildi: SVC, Random Forest, Naive Bayes, Nearest Centroid ve XGBoost. Ayrıca bütün modellerin tahminlerinden en sık çıkanı seçen bir topluluk (bagging tarzı çoğunluk oylaması) seçeneği vardı.

Arayüzde veri setini bölme, modeli yeniden eğitme, parametre girme ve test etme seçenekleri vardı. Test sonucunda F1 skoru ve doğruluk görünüyordu. Kullanıcı kendi yazdığı bir metni de sınıflandırabiliyordu. Demoda görülen doğruluklar şöyleydi:

Model	Doğruluk (demoda söylenen)
XGBoost	%97
Random Forest	%97
Naive Bayes	%97
SVC	%87
Nearest Centroid (BoW)	en fazla %60
Nearest Centroid (TF-IDF)	%92

Grup, Nearest Centroid'in TF-IDF ile çok daha iyi sonuç vermesini TF-IDF'in kelimelerin önemini hesaba katmasıyla açıkladı.

10.2 Kod Yapısı

Flask uygulamasında bir model sözlüğü (model dictionary) basit BoW ve TF-IDF BoW için modelleri yüklüyordu. Grubun kendi yazdığı bir modülde iki model sınıfı vardı. İkisinde de `fit`, `predict`, `evaluate` ve `get_params` fonksiyonları tanımlıydı. Böylece iki model aynı arayüzle kullanılabiliyordu. JavaScript tarafı, örneğin "SVC, Basic, BoW" gibi seçimleri istek olarak gönderiyordu. Sayfanın yenilenmemesi için AJAX istekleri kullanıldı. Metin ön işlemede özel karakterler temizleniyordu.

10.3 Docker ve Kubernetes

Uygulama Docker ile paketlendi. Dockerfile bir Ubuntu imajı kuruyor, Python'u ve gereksinim dosyasındaki kütüphaneleri yüklüyor, sonra Flask'ı çalıştırıyordu. Bazı kütüphaneler bu yolla kurulamadığı için ayrıca pip ile kuruldu. Docker Compose dosyasında uygulamanın hangi portta çalışacağı ve yerelden hangi porttan bağlanılacağı belirtildi. Aynı imaj Google Cloud'a aktarıldı. Orada Kubernetes, toplam yaklaşık 12 GB RAM ayrılmış üç makine üzerinde kapsayıcıları (container) kaynaklara dağıtıp çalıştırıyordu.

11. Operatör Yorumlarında Kümeleme

Onuncu grup bir mobil operatöre 2019 yılında yapılmış olumlu, olumsuz ve nötr yorumlardan oluşan bir veri setiyle çalıştı. Öneri belgesinde K-means önermişlerdi. Hoca öneri aşamasında DBSCAN ve Gaussian Mixture Model'i (GMM) de kullanmalarını istemişti. Grup arayüz yapmadığını ve yalnız K-means'i kullanabildiğini söyledi.

Ön işleme adımları şunlardı:

- Harfleri küçültmek ve metni kelimelere ayırmak.
- Bazı özel karakterleri kaldırmak.
- "ve", "veya" gibi Türkçe durak kelimeleri (stop words) çıkarmak.
- Kelimeleri köklerine indirmek (lemmatization) ve cümleyi yeniden birleştirmek.

K'nın 1'den 9'a kadar değerleri için bir grafik çizildi. Hoca "K'yı ne seçtiniz?" diye sordu. Grup, veriye göre en uygun K'yı otomatik seçen bir kütüphane bulduklarını söyledi. Hoca kütüphanenin arka planda nasıl çalıştığını sordu. Grup yalnız ne yaptığını bildiğini, nasıl yaptığını bilmediğini söyledi.

> **Hocanın vurgusu:** Bir kütüphanenin yalnız ne yaptığını değil, nasıl yaptığını da öğrenmeniz gerekir.

Sonuçlarda 0 olumsuz, 1 olumlu, 2 nötr olarak kodlandı. Olumsuz etiketli yorumların bir kısmı başka kümelere düşmüştü. Olumlu ve olumsuz kümeler için kelime görselleri çıkarıldı. Operatörün kendi adı iki kümede de çok sık geçiyordu ve aslında duygu taşımıyordu, ama silinememişti. Olumsuz kümede rakip operatörlerin adları gibi kelimeler yakalanmıştı.

12. Tweet Duygu Analizi (Logistic Regression)

On birinci grup tweet duygu analizi yaptı. Öneri belgesinde Naive Bayes ve Decision Tree yazıyordu. Grup bunlarla düşük doğruluk aldığı için Logistic Regression'a geçtiğini, Random Forest'ı da denediğini söyledi. Denedikleri modeller kodda yoktu.

Hocanın vurgusu: En az üç algoritma istendi. Diğerlerini denediyseniz bunu kodda göstermeniz gerekir. Gösterilmediğinde yalnız "denedik" sözü kalır.

12.1 Veri ve Demo

Eğitimde toplam 28 bin tweet kullanıldı: NLTK'nın `twitter_samples` derleminden 8 bin tweet ve yetersiz kaldığı için Kaggle'dan 20 bin tweet ($8000 + 20000 = 28000$). `twitter_samples` içinde 5 bini olumlu, 5 bini olumsuz 10 bin tweet vardı. 8 bini eğitime, kalan 2 bini teste ayrıldı. Test doğruluğu %98 olarak söylendi. Arayüzde üç düğme vardı: eğitim, test ve tahmin. "I am happy" olumlu, "I am not happy" olumsuz tahmin edildi.

Hoca test sonucunda karışıklık matrisi ya da başka bir ölçü olup olmadığını sordu. Yalnız doğruluk yazdırılmıştı.

12.2 Ön İşleme ve Özellikler

Tweet ön işleme şöyleydi:

- Dolar işareti gibi para birimi simgelerini kaldırmak.
- Retweet'lerin başındaki "RT" işaretini kaldırmak.
- Bağlantıları kaldırmak.
- Hashtag işaretini kaldırmak, ama arkasındaki kelimeyi bırakmak. Çünkü o kelime de duygu taşıyabilir.
- Tokenizer ile kelimelere ayırmak, durak kelime ve noktalama değilse kökünü almak.

Önemli bir ayrıntı: Durak kelime listesinde "not" gibi duygu yönünü değiştiren kelimeler de var. Bunlar atılınca "I am not happy" olumlu çıkıyordu. Grup bu kelimeleri frekans hesabına dahil edince doğruluğun arttığını söyledi.

Özellik çıkarma (`extract_features`) her tweet için bir frekans vektörü oluşturuyordu. Bir sözlük (`build_freqs`) her kelimenin kaç olumlu ve kaç olumsuz tweette geçtiğini tutuyordu. Tweetin özellikleri, kelimelerinin olumlu ve olumsuz tweetlerdeki toplam kullanım sayılarıydı. Model scikit-learn'ün Logistic Regression'ıyla `fit` edildi ve `pickle` ile kaydedildi. Flask arayüzü kaydedilmiş modeli yükleyip tahmin 1 ise "pozitif", 0 ise "negatif" yazıyordu.

13. Kullanıcı ve Öğe Tabanlı Film Önerisi (k-NN)

On ikinci grup işbirlikçi filtreleme (collaborative filtering) ile bir film öneri sistemi yaptı. Grup iki türü şöyle açıkladı:

- **Kullanıcı tabanlı (user-based):** Bir kullanıcıya benzeyen kullanıcılar bulunur. Komşuların izleyip bu kullanıcının izlemediği filmler önerilir.
- **Öğe tabanlı (item-based):** Filmler üzerinden gidilir. "Bu ürünü alanlar şu ürünü de aldı" mantığıdır.

Arayüz tkinter ile yazılmıştı. Kodun akışı şöyleydi:

1. Kullanıcıların puanları ve ayrı bir dosyadaki film adları okunup birleştirildi (merge). Tür bilgisi kullanılmadı.
2. 30'dan az oy alan filmleri ve 30'dan az oy veren kullanıcıları atan bir süzme yazılmıştı. Sonuçları kötü etkilediği için yorum satırına alındı.
3. Veri eğitim ve test olarak bölündü, pivot tablo ile kullanıcı-film matrisine çevrildi.
4. **Kullanıcı tabanlı kısım:** k-NN ile $k = 25$ komşu, uzaklık ölçüsü olarak kosinüs kullanıldı. Komşuların izlediği filmler bir sözlükte toplandı. Kullanıcının izlemediği ve birden fazla komşunun izlediği filmler seçildi. Puanlar benzerliklerle çarpılıp toplandı ve benzerlik toplamına bölündü.
5. **Başarı ölçümü:** Test verisinde kullanıcının izlediği filmler için tahmin edilen puanla gerçek puan karşılaştırıldı. Ayrıca bir hata ölçüsü yazdırıldı.
6. **Öğe tabanlı kısım:** Hazır kütüphane kullanıldı. Matris çok seyrek (sparse) olduğu için boyut indirgeme (dimensionality reduction) yapılıp filmler arası benzerliklere göre sıralandı.

14. Reddit Yorum Kaldırma Tahmini

On üçüncü sunumdaki proje Reddit'teki bir bilim alt grubunun (subreddit) verisine dayanıyordu. Her alt grubun kendi kuralları vardır ve bunları moderatörler uygular. Günde binlerce gönderi olan büyük gruplarda yorumları elle denetlemek zordur, bu yüzden otomatik moderatör botları kullanılır. Projenin amacı, bir yorumun kaldırılması gerekip gerekmediğini söyleyen bir API sunmaktı. Hoca bunun bir metin sınıflandırma problemi olduğunu belirtti.

Dört tür sınıflandırıcı kullanıldı, sonuncusunun üç alt türü vardı:

- Logistic Regression.
- Bir makaleden öğrenilen Naive Bayes ile SVM'yi birleştiren yaklaşım (NB-SVM).
- Dengesiz verilerde iyi bir temel (baseline) olduğu için Random Forest.
- spaCy'nin metin sınıflandırma hattı: basit kelime torbası, basit bir CNN ve ikisinin birleştirildiği (stacking) mimari.

Demo arayüzü yalnız gösterim içindi. Asıl amaç API'ydi. Yanıtta işlemin başarılı olup olmadığı, kaldırılma ve kaldırılmama olasılıkları ve tüketicinin olasılıkları yeniden kontrol etmesine gerek bırakmayan bir karar alanı dönüyordu.

Kod yapısı şöyleydi:

- API ve şablon motoru FastAPI ile yazıldı.
- **Metin temizleme dönüştürücüsü:** Satır başı karakterlerini ve Markdown biçimlendirmesindeki noktalama işaretlerini kaldırıyor, metni yeniden kodluyordu. Özgün veri ISO-8859-1 kodlamasındaydı. Veri UTF-8'e, olmazsa CP1252'ye çevriliyordu.
- **spaCy token dönüştürücüsü:** N-gram ya da regex yerine spaCy'nin sağlam tokenizasyon hattı kullanıldı. Durak kelime, yasaklı simge ya da noktalama olmayan tokenler döndürülüyordu.
- Bu iki adımı birleştiren bir metin ön işleyici, scikit-learn ve spaCy modelleri için yardımcı sınıflar.
- **Model yükleyici:** Yapılandırma dosyasını okuyup model dizinindeki bütün modelleri yüklüyordu. API'ye istek geldiğinde tahmin yapılıp sonuç döndürülüyordu.
- Uygulama, asenkron bir ASGI sunucusu (Uvicorn) üzerinde Gunicorn ile yayınlanıyordu.

Hoca projeyi çok iyi bulduğunu söyledi.

15. E-posta Spam Tespiti

On dördüncü grup bir e-posta spam tespit uygulaması yaptı. Arayüzde dört veri setinden ve beş algoritmadan biri seçiliyordu. Veri setlerinin üçü Kaggle'dan, biri başka bir kaynaktan alınmıştı. Kullanıcı ya test verisinden bir satır numarası giriyor ya da kendi yazdığı bir e-postayı veriyordu. Sistem accuracy, recall, precision ve F1 skorunu yazdırıyor, girilen metnin spam olup olmadığını söylüyordu.

Ön işleme ve özellik çıkarma şöyleydi:

1. Nokta, virgül, soru işareti gibi işaretler temizlendi.
2. Cümleler tokenize edilip kelimelere ayrıldı.
3. Veri setinde en sık geçen 1000 kelime tutuldu.
4. Her cümle için bu 1000 kelimenin her biri cümlede varsa 1, yoksa 0 yazılarak ikili (binary) bir vektör oluşturuldu.
5. Veri %80 eğitim, %20 test olarak bölündü.

Veri setleri arasındaki fark yalnız sütun adlarıydı. Ön yüz React'in Next.js çatısıyla yazıldı. Next.js'te ön yüz ve arka yüz birlikte olduğu için arka yüzde bir HTTP isteği seçilen veri setini, algoritmayı, satırı ve e-postayı alıyordu. Arka yüz `spawn` komutuyla Python betiğini çalıştırıyor, betiğin `print` ile yazdığı JSON'u yanıt olarak döndürüyordu.

16. Kitap Fiyat Karşılaştırma Tarayıcısı

On beşinci grup üç kitap sitesinden (D&R, idefix ve Kitapyurdu) kitap bilgisi çekip fiyat karşılaştırması yapan bir uygulama sundu. Veritabanı olarak Google Firestore kullanıldı, ön yüz

Heroku'da yayınlandı. Kitap adı girilince bütün sitelerdeki fiyatlar listeleniyor, tıklanınca ilgili siteye gidiliyordu.

Hoca arama hakkında sorular sordu. Yalnız kitap adıyla arama yapılabiliyordu, yazar adıyla yapılamıyordu. Kitap adının birebir aynı yazılması gerekiyordu. Grup bunun nedenini Firestore'da SQL'deki `LIKE` gibi bir sorgunun olmamasıyla açıkladı. Hoca, sorguyu ve kitap adlarını vektöre çevirip kosinüs benzerliğiyle eşleştirip eşleştirmediklerini sordu. Grup doğrudan veritabanında arama yaptıklarını söyledi.

Tarayıcı Node.js ve Puppeteer ile yazıldı. Her site için kitapların adı, fiyatı, yazarı ve görseli çekilip bir nesneye dönüştürülerek veritabanına gönderiliyordu. Heroku'da her 24 saatte bir çalışan bir tarayıcı vardı. Firestore'un ücretsiz sürümünde günlük yaklaşık 20 bin yazma sınırı olduğu için Kitapyurdu'nda yaklaşık 1000 sayfa yerine yalnız 85 sayfa tarandı.

17. Akademik Makale Başlığı Üretme (LSTM ve T5)

On altıncı grup akademik makaleler için bir başlık öneri sistemi yaptı. Girdi makalenin özeti (abstract), çıktı üç farklı başlık önerisiydi. Grup yaptıkları işin aslında bir özetleme (summarization) problemi olduğunu söyledi.

17.1 Tarayıcı ve Veri

Grup arXiv sitesinin gelişmiş arama aracını tarayan bir tarayıcı yazdı. Tarayıcı komut satırından üç argüman alıyordu: arama yapılacak alan (örneğin doğal dil işleme için `cs.CL`), başlangıç indeksi ve bitiş indeksi. URL'nin bu parçaları değiştirilerek sayfalar geziliyor, her makalenin başlığı ve özeti alınıyordu. Bu yolla NLP alanında 0 ile 5000 indeksleri arasındaki makaleler tarandı. Model eğitimi için bu veri yeterince kapsamlı olmadığından, Cornell Üniversitesi'nin arXiv verisinden oluşan hazır bir Kaggle veri seti kullanıldı. Bu veri setindeki yayın yeri ve yazarlar gibi üst veriler (metadata) atıldı, yalnız başlık ve özet kaldı. Grup kendi regex tokenizer'larını yazdı ve keşifsel veri analizi (exploratory data analysis) grafikleri hazırladı.

17.2 Modeller

İki model kullanıldı:

- **Temel model (baseline):** Kelime gömme katmanı (embedding layer) olan bir sequence-to-sequence LSTM. Keras ile yazıldı. Bir kodlayıcı (encoder) ve bir kod çözücü (decoder) LSTM'den sonra hedef sözlük boyutuna eşleyen bir yoğun (dense) katman vardı. RMSprop optimizasyonu ve kategorik çapraz entropi (categorical cross-entropy) kaybı kullanıldı. Kelime-indeks ve indeks-kelime sözlükleri tanımlandı, indeksler en uzun dizi uzunluğuna göre yığınlara (batch) bölündü. GloVe vektörleri eklenmiş iki sürüm de yazıldı ama eğitmeye zaman kalmadı. LSTM yaklaşık 7 saat eğitildi.
- **T5:** Google'ın metinden metne dönüştürücü (Text-to-Text Transfer Transformer) modeli. Hugging Face'in Transformers kütüphanesiyle T5-base kullanıldı. Model Colab'da yaklaşık bir hafta, kesintilerle eğitildi. Ara kayıtlar (checkpoint) Google Drive'a kaydedildi ve açık kaynak olarak paylaşıldı. BERT ve BART da denenmiş, en iyi sonucu T5 vermişti.

Grup başta BLEU skoru ve perplexity kullanmayı planlamış, ama zaman kalmadığı için doğruluk ve kayıp (loss) değerleriyle yetinmişti. İkisi de yakınsamıştı. Ön işleme bilerek az tutuldu: Abstract tokenizer yalnız durak kelimeleri ve sayıları atıyordu. Daha fazla ön işleme anlam kaybına yol açıyor ve sonuçları iyileştirmiyordu. Ön işlemesiz deneme daha iyi sonuç verdi.

17.3 Demo

Flask ile bir arayüz yazıldı. LSTM başlık biçimini (örneğin "yöntem adı: açıklama") öğrenebiliyor, ama farklı alanlarda ortak kelimeler görünce karışıyordu. Örneğin birinci dereceden gradyan optimizasyonuyla ilgili bir özete süpernova ile ilgili bir başlık önerebiliyordu. Word2Vec makalesinin özeti için LSTM çok genel bir başlık üretti. T5 ise sürekli skip-gram modeli ve vektör temsilleriyle ilgili, konuyu yakalayan başlıklar önerdi. Son olarak hocanın IoT sistemlerinde kendi kendini onarma (self-healing) konulu bir makalesinin özeti verildi ve model "runtime verification mechanism" ifadesini içeren bir başlık önerdi. Hoca ikinci öneriyi daha uygun buldu ve projeyi beğendiğini söyledi.

> **Ders sonrası not:** Grup T5'in 2018'de çıktığını ve modelin yaklaşık 500 milyon parametresi olduğunu söyledi. T5 makalesi büyük olasılıkla 2019'da yayımlandı ve T5-base büyük olasılıkla yaklaşık 220 milyon parametreye sahiptir. Ayrıca grup Keras'taki gömme katmanını "kelimelerin yanındaki kelimelerle olasılığını maksimize eden" bir katman diye anlattı. Bu tanım Word2Vec'in skip-gram eğitim amacına uyar. Modelle birlikte eğitilen bir Keras `Embedding` katmanı ise modelin kendi kayıp fonksiyonuna göre güncellenen bir arama tablosudur.

18. Web Sunucusu Log Analizi ve Birliktelik Kuralları

On yedinci sunum tek kişilik bir log analizi aracıydı. Kullanıcı verilerini görmek istediği için bu konuyu seçen öğrenci Python kullandı. Veri seti, ABD kaynaklı bir sitenin halka açık yayımladığı web sunucusu loglarıydı. Bir günlük log yaklaşık 2,5 GB olduğu için dosyalar kısaltılarak kullanıldı. Program, Python kurulu olmayan bilgisayarda da çalışsın diye çalıştırılabilir dosyaya (exe) çevrilmişti.

Araç şu istatistikleri gösteriyordu:

- Doküman türleri ve kullanım sayıları (19 farklı doküman türü), çubuk ve pasta grafikleri.
- Erişilen toplam dosya boyutu (yaklaşık 23 GB).
- Erişim kodları: başarılı, yönlendirme ve hata. Yaklaşık 164 bin başarılı ve 10 bin hatalı erişim vardı.
- Zamana göre erişim yoğunluğu.

Hoca bunların daha çok log dosyasındaki istatistikler olduğunu söyledi ve birliktelik algoritmaları ya da sıralı örüntü madenciliği (sequential pattern mining) kullanılıp kullanılmadığını sordu. Öğrenci birliktelik kuralı madenciliği (association rule mining) için Apriori algoritmasını kullanmıştı.

Kodda loglar pandas'ın `read_table` fonksiyonuyla okunuyor ve IP adresine göre oturumlara (session) ayrılıyordu. Her oturumda bir kullanıcının sırayla eriştiği doküman kimlikleri bir işlem (transaction) oluyordu. Yaklaşık 45 bin işlem vardı. Kullanıcı en düşük destek (minimum support) ve güven (confidence) değerlerini giriyordu. İşlem sayısı büyük olduğu için destek değeri 0,01 ile 0,001 arasında olmalıydı, 0,01'in üstünde hiçbir kural çıkmıyordu. Apriori bir Python kütüphanesinden alındı ve kuralı sağlayan doküman kimlikleri ekrana basıldı.

Yaklaşık 45 bin işlemde destek eşiğinin anlamı şöyledir (ara adım eklendi):

$$0{,}01 \times 45000 = 450, \qquad 0{,}001 \times 45000 = 45$$

Yani destek 0,01 olduğunda bir doküman kümesinin en az yaklaşık 450 oturumda birlikte geçmesi gerekir.

> **Hocanın vurgusu:** Birden fazla birliktelik kuralı algoritmasını kullanıp karşılaştırmalı incelemek projeyi daha iyi yapardı.

Hoca kodların ayrıca, readme dosyasıyla birlikte ve exe'ye nasıl dönüştürüldüğü belirtilerek proje olarak gönderilmesini istedi.

19. Gazete Haberlerinde Taraf Belirleme

On sekizinci grup bir fikir madenciliği projesi yaptı. Konu, son dönemde yaşanan bir uluslararası silahlı çatışma hakkında Avrupa ve çevresindeki ülkelerin gazetelerinin hangi tarafı desteklediğini bulmaktı. Grup, hükümetlerin bakış açılarını gazeteler aracılığıyla yansıttığı varsayımından yola çıktı. Bakılan ülkeler yalnız AB ülkeleri değildi, çevredeki başka ülkeleri de kapsıyordu. Demoda yaklaşık 40 ülke için her tarafa destek oranları gösterildi. Oranları birbirine yakın ülkeler nötr kabul ediliyordu.

19.1 Veri ve Model

Grup çeşitli ülkelerden yaklaşık 280 haber ve dışişleri bakanlığı açıklamasını elle topladı ve etiketledi. Her ülkeden en az 5 metin vardı. İçeriği bol olan ülkelerden 20'den fazla metin

toplanmıştı. Hoca veriyi toplamak için bir tarayıcı yazıp yazmadıklarını sordu. Grup, verinin sonunda etiketlenmesi gerektiğini ve daha önce hazırlanmış örnek bir veri seti olmadığını söyledi. Büyük miktarda veriyi tarayıp otomatik etiketleme şansları yoktu, bu yüzden doğru veriyi seçmek için elle topladılar.

Model .NET'in ML.NET kütüphanesiyle kuruldu. Bire karşı hepsi (one-versus-all) yaklaşımıyla çok sınıflı sınıflandırma yapıldı ve algoritma olarak destek vektör makinesi (SVM) kullanıldı. Tahmin için üç girdi veriliyordu: ülkenin iki harfli ISO kodu (alpha-2), haberin başlığı ve gövdesi. Başlık ve gövde boş bırakılıp yalnız ülke kodu verildiğinde, modelin o ülkenin hangi tarafa daha yatkın olduğunu söylemesi amaçlanıyordu.

19.2 Geri Bildirim Döngüsü

Model tahmin yaptıktan sonra uygulama tahminin doğru olup olmadığını soruyordu. Kullanıcı doğruysa onaylıyor, yanlışsa doğru etiketi giriyordu. Sonra örnek veri setine eklenebiliyordu. Konsoldan model yeniden eğitilebiliyordu. Dosyadan toplu veri girme de vardı: Her metin için tahmin gösteriliyor ve modelin skoru, metni tamamen okumadan etiketlemeyi hızlandırmak için kullanılabiliyordu.

Hoca bunun örnek tabanlı (instance-based) bir öğrenme olup olmadığını sordu. Grup, yeniden eğitimde bütün veri setinin baştan, sıfırdan modele verildiğini gösterdi.

Veri temizleme fonksiyonu şunları yapıyordu:

- Tekrarlanan kayıtları bulmak. Tekrar yalnız gövdeye göre belirleniyordu: aynı başlıklı haberlere izin veriliyor, aynı gövdeli haberlere verilmiyordu.
- Başlığı ya da gövdesi boş kayıtları bulmak.
- Web'den kopyalanırken bozulan özel karakterleri (örneğin ters tırnak yerine görünen üç karakterlik diziler) bir LINQ sorgusuyla silmek.

19.3 Hocanın Özellik Çıkarma Sorusu

Hoca, kelimelerden oluşan bir cümlenin SVM'ye nasıl bir vektör olarak verildiğini sordu. Grup özellik çıkarma yapmadıklarını, ML.NET'in metin sınıflandırma sırasında bunu arka planda otomatik yaptığını söyledi. Kendileri yalnız özellik sütunlarını ve etiket sütununu belirtiyordu. İterasyon sayısını anlatınca hoca sorunun başka olduğunu söyledi. Örnek olarak bir özellik çıkarma yöntemi verdi: Word2Vec ile cümledeki her kelime bir vektöre çevrilir, cümledeki kelime vektörlerinin ortalaması alınır ve bu ortalama vektör SVM'ye verilir. Başka bir vektörleştirme tekniği de olabilir. Önemli olan, kütüphanenin arka planda cümleyi nasıl bir vektöre çevirdiğini öğrenmekti. Grup bunu araştırmadığını, birkaç model deneyip en başarılısını seçtiklerini söyledi. Hoca fikri ve çalışmayı güzel bulduğunu, arayüze daha fazla özen gösterilebileceğini söyledi.

> **Ders sonrası not:** Grup bir tahminde modelin verdiği 1,75 değerini "%175" diye yorumladı. Bire karşı hepsi yaklaşımında ML.NET'in sınıflar için döndürdüğü değerler büyük olasılıkla olasılık değil, karşılaştırma için kullanılan ham skorlardır. Bu yüzden yüzde olarak okunamazlar. En yüksek skorlu sınıf tahmin edilen sınıftır.

20. Ayakkabı Fiyat Tarayıcısı ve Filtreleme Eksikliği

On dokuzuncu grup ayakkabı satan iki e-ticaret sitesinden ürün çekip fiyatları karşılaştırdı. Başta tek site planlanmıştı, hocanın isteğiyle ikinci site eklendi. Uygulama iki komutlu bir komut satırı aracıydı. Birinci komut bütün ürünleri çekip CSV'ye kaydediyordu. İkinci komut kaydedilen veriden toplam ürün sayısını, ortalama fiyatı, en ucuz ve en pahalı ürünleri veriyordu.

Kod Python 3.6 ve 3.7'de çalışıyordu. `json`, `csv` ve `sys` standart modüllerinin yanında iki üçüncü taraf kütüphane kullanıldı: HTTP istekleri için `requests` ve CSV/Excel işlemlerini kolaylaştırdığı için `pandas`. Bir istemci sınıfında şunlar vardı:

- Siteleri dinamik tutmak için bir sözlük.
- Siteye istek atıp gelen JSON yanıtını sözlük olarak döndüren bir yardımcı metot.
- Bir sitede kaç ürün, kaç sayfa ve sayfa başına kaç ürün olduğunu döndüren bir metot.

- Verilen site ve sayfa numarasına göre ürünleri döndüren bir metot. Ürünlerin birçok alanından dördü alındı: ad, indirimli fiyat ve kendi fiyatı dahil.
- Bütün veriyi birleştirip her site için ayrı bir CSV dosyası oluşturan kısım.

Hoca gelen ürünlerin fiyata göre filtrelenip filtrelenemediğini sordu. Grup bütün ürünleri CSV'ye yazdığını, filtrelemeyi yerelde yapmayı düşündüğünü, ama arayüzde böyle bir özellik olmadığını söyledi. Arayüzün de konsol olduğunu belirtti.

> **Hocanın vurgusu:** Bir e-ticaret sitesine kullanıcı olarak gittiğinizde bütün ürünleri görmek istemezsiniz. Örneğin yalnız Nike ürünlerini ya da yalnız 43 numara ayakkabıları görmek istersiniz. Birden fazla siteden gelen veriyi kategori ve fiyat bazında sıralamak ve süzebilmek gerekir. Projeyi kabul ederken istenen buydu ve bunu çok iyi yapan gruplar vardı.

Kodlar henüz gönderilmemişti. Hoca kodların, nasıl çalıştırılacağını anlatan bir readme dosyasıyla birlikte aynı gün gönderilmesini istedi.

21. Şarkı Öneri Sistemi ve Değerlendirme Eksikliği

Yirminci sunum tek kişilik bir şarkı öneri sistemiydi. Öğe tabanlı ve kullanıcı tabanlı öneri yapılabiliyordu. Öğe tabanlı demoda bir şarkı kimliği ve gösterilecek satır sayısı girildi. Sistem en benzer şarkıları, kaç kişinin dinlediğini, ortalama puanını ve benzerlik değerini gösterdi.

Kodun akışı şöyleydi:

1. Veri seti Jupyter'de yüklendi ve puanlardan bir pivot tablo (kullanıcı-şarkı matrisi) oluşturuldu.
2. Öğe tabanlı fonksiyon matrisi ve şarkı kimliğini alıp o şarkının sütununu ayırdı. Diğer şarkılarla korelasyon katsayısı hesaplandı.
3. Tanımsız (null) benzerlikler atıldı ve sonuç sıralandı.
4. `groupby` ve toplama (aggregate) fonksiyonlarıyla her şarkının kaç kişi tarafından dinlendiği ve ortalama puanı bulundu.
5. Yalnız 100'den fazla kişinin dinlediği şarkılar tutulup benzerlik tablosuyla birleştirildi (join).

Hoca yapılan önerilerin ne kadar başarılı olduğunun değerlendirilip değerlendirilmediğini sordu. Öğrenci değerlendirme yapmadığını söyledi. Hoca, gösterilenin yalnız seçilen şarkı ile veritabanındaki şarkılar arasındaki benzerlik olduğunu, sonuçlar için ayrıca bir değerlendirme yöntemi gerektiğini belirtti. Bu yapılmadığında önerilerin iyi mi kötü mü olduğunu bilemeyiz.

> **Ek bilgi (derste anlatılmadı):** Öneri sistemleri genellikle şöyle değerlendirilir: Kullanıcıların puanlarının bir kısmı ayrılır, model bu puanları tahmin eder ve tahminle gerçek puan arasındaki hata RMSE ya da MAE ile ölçülür. Liste biçimindeki önerilerde ise ilk k öneri içinde kullanıcının gerçekten beğendiği öğelerin oranı precision@k ve recall@k ile ölçülür.

22. Hazır Kütüphaneyle Tweet Duygu Analizi

Yirmi birinci grup bir Twitter duygu analizi aracı yaptı. Arayüz C# ile Windows masaüstü uygulaması olarak tasarlanmıştı ve Python kodunu çağırıyordu. Kullanıcı bir anahtar kelime giriyor, Twitter API'si bu konudaki tweetleri döndürüyordu. Tweetler bir Excel dosyasına kaydedilip tabloda gösteriliyordu. Her tweetin yanında olumlu, olumsuz ya da nötr sınıfı, sağ tabloda da yüzdeler vardı.

Hoca sınıfların neye göre belirlendiğini sordu. Grup TextBlob kütüphanesinin duygu fonksiyonunu kullandıklarını, bir makine öğrenmesi modeli eğitmediklerini söyledi. Gerekçe olarak API'nin bir seferde yalnız 70-80 tweet verdiğini ve hazır bir Twitter veri seti bulamadıklarını söylediler.

Hoca bunu kabul etmedi. Arka planda hangi modelin kullanıldığının tam olarak bu projenin işi olduğunu söyledi. İzlenmesi gereken yol şuydu:

1. Cümleleri örneğin Word2Vec ile vektöre dönüştürmek.
2. Vektörleri etiketlerle birlikte etiketli veri hâline getirmek.

3. Bu veriyle birbirinden farklı üç makine öğrenmesi algoritmasını eğitip karşılaştırmak.

Hoca az veriyle de proje yapılabileceğini, bir grubun 500 etiketli veriyi elle oluşturduğunu örnek verdi.

> **Hocanın vurgusu:** Hazır bir kütüphane fonksiyonunu çağırmak, derste öğrenilenleri bir araca yaptırmak demektir. Sınıflandırma projelerinde kendi etiketli verinizle en az üç farklı algoritma eğitmeniz gerekir. Proje iki ay önceden verildi ve öneri belgesinde neyin ne zaman yapılacağı yazılıydı. "Başka derslerin projeleri vardı" gerekçesi geçerli değildir.
>
> **Ders sonrası not:** Grup TextBlob sonucunun "sıfıra eşitse pozitif, değilse negatif, hiçbiri değilse nötr" diye yorumlandığını söyledi. TextBlob'un duygu fonksiyonu büyük olasılıkla $[-1, 1]$ aralığında bir kutupluluk (polarity) değeri döndürür. Yaygın kullanımda değer sıfırdan büyükse olumlu, sıfırdan küçükse olumsuz, sıfıra eşitse nötr kabul edilir.

23. Web Sayfasından Anahtar Kelime Çıkarma

Yirmi ikinci sunum tek kişilik bir içerik anahtar kelimesi bulma aracıydı. Kullanıcı bir URL veriyor, uygulama sayfayı tarayıp içinden anahtar kelimeleri buluyor ve sonucu SQLite'a kaydediyordu. Daha sonra kayıtlı anahtar kelimelerle arama yapılabiliyordu. Örneğin bir sayfadan "MVP" anahtar kelimesi bulunmuştu. "MVP" diye arayınca bu etiketle eşleşen site listeleniyordu. Uygulama Heroku'da yayınlanmıştı.

Kodun akışı şöyleydi:

1. Tarayıcı modülü sayfayı alıyor, bir HTML ayrıştırıcıyla (parser) yalnız gövdedeki (body) etiketleri silip bütün metni alıyordu.
2. Bir anahtar kelime bulucu sınıfı, NLTK'nın durak kelimelerini kullanan RAKE yöntemiyle metinden anahtar kelimeleri çıkarıp puanlıyordu.
3. Dönen dizi en anlamlıdan en az anlamlıya sıralıydı. Sıfırıncı eleman, yani en yüksek puanlı anahtar kelime alınıp veritabanına kaydediliyordu.

Hoca projenin en önemli kısmının metinden anlamlı kelimeleri çıkarmak olduğunu belirtti ve bunun nasıl yapıldığını sordu. Öğrenci bu kısmı hazır kütüphaneden aldığını kabul etti.

> **Hocanın vurgusu:** Derste işlediğimiz şeyleri sizin kodunuzun içinde görmek istiyorum.

24. Telefon Yorumları Tarayıcısı (Scrapy, MongoDB, React)

Yirmi üçüncü grup Hepsiburada ve Trendyol'dan telefon ürünlerini ve yorumlarını çeken bir uygulamayı Heroku'da yayınladı. En az bir yorumu olan bütün telefonlar, yorumlarıyla birlikte taranmıştı. Ürün sayfasında ürün bilgileri, özgün siteye bağlantı, öne çıkan yorum ve diğer yorumlar vardı. Yorumlar sitelerin kendi popülerlik sıralamasıyla gösteriliyordu.

Filtreler fiyat aralığı, en düşük puan, değerlendirme sayısı ve yorum sayısıydı. Sonuçlar puana göre sıralanabiliyordu. Demoda 5000 ile 10.000 lira arası ve 3 puan üzeri telefonlar süzüldü.

24.1 Statik ve Dinamik Sayfaları Tarama

Tarayıcılar Python'da Scrapy ile yazıldı. İki site iki farklı yaklaşım gerektirdi:

- **Hepsiburada:** Statik HTML sayfası kullanıldı. Bilgiler etiketlerden çekildi. Sayfalama (pagination) olduğu için en alttaki sayfa sayısı alınıp her sayfa ayrıştırıldı.
- **Trendyol:** Sayfa dinamikti. Sayfalama yoktu, aşağı indikçe ürünler yükleniyordu. Bu yüzden tarayıcının ağ (network) sekmesinden sitenin REST API'si bulundu. Bu bağlantı izlenerek bütün bilgiler JSON nesnelerinden çekildi. Yorumlar da dinamik yüklendiği için aynı yöntemle alındı.

JSON yanıtları Python sözlüğüne çevrilip önce ürünler, sonra her ürünün yorumları ayrıştırıldı. Scrapy'de her öğe `yield` edildiğinde boru hattı (pipeline) köprü görevi görüp öğeyi Mon-

goDB'ye yazıyordu. Grup MongoDB'yi seçme nedenini iki şekilde açıkladı: Büyük veri alanında NoSQL'in önerilmesi ve verinin zaten JSON/sözlük biçiminde gelmesi.

24.2 RESTful API ve Ön Yüz

Arka yüz Flask ile yazılmış bir RESTful API'ydi. Arama kriterlerine uyan ürünleri ve tek bir ürünü döndüren uç noktalar vardı. Grup REST'i seçme nedenini veri büyüklüğüyle açıkladı. Ön yüz React ile statik bir sayfa olarak yazıldı ve yalnız değişen kısımlar API'den çekildi. Böylece HTML ile birlikte büyük veri taşınmadı ve sayfa daha hızlı yüklendi. Hoca projeyi güzel bulduğunu söyledi.

25. LG Televizyon Fiyat Tarayıcısı (Scrapy ve Flask)

Yirmi dördüncü grubun öneri belgesinde PlayStation satan sitelerden veri çekmek yazıyordu. Grup bunu daha önce hocaya sorup ürünü LG televizyon modelleri olarak değiştirmişti. Hoca ürün değişikliğinde sorun olmadığını söyledi.

Dört siteden (Evkur, Teknosa, Vatan Bilgisayar, GittiGidiyor) veri çekildi. Arayüzde ürün fotoğrafı, adı ve fiyatı listeleniyordu. Filtreler site, fiyat aralığı ve ekran boyutuydu (örneğin 43, 49, 50 inç). Ürüne tıklayınca satıldığı sayfaya gidiliyordu.

25.1 Scrapy Kodu

`scrapy startproject` ile proje oluşturuldu ve her site için ayrı bir örümcek sınıfı yazıldı. Örümcek `name` alanındaki adla çalıştırılıyordu, örneğin `scrapy crawl gittigidiyor -o lgtv.json`. Başlangıç URL'lerindeki bağlantılar tek tek taranıp gelen HTML `parse` fonksiyonuna gönderiliyordu. `parse` içinde ürün adı, fiyat, görsel ve ürün sayfası `response.xpath` ile ilgili sınıflar gezilerek çekildi.

Bazı site özel sorunları çözülmesi gerekti:

- GittiGidiyor'da fiyat listesine boşluk elemanları da geliyordu. Boşluk elemanının uzunluğu 17 olduğu için bu uzunluktaki elemanlar listeden çıkarıldı.
- GittiGidiyor'da ürün sayfası bağlantısının yapısı farklıydı. Başında site adı vardı. Bağlantı `split` ile ayrılıp yalnız gereken parçalar alındı ve araya eğik çizgi eklenerek diğer sitelerle aynı yapıya getirildi.

Alanlar `zip` ile birleştirilip bir ham veri listesinde tutuldu. Döngüyle her ürün için ad, fiyat, görsel ve sayfa alanlarından oluşan bir sözlük oluşturulup `yield` ile döndürüldü. Sonuç JSON ve CSV dosyalarına yazıldı.

25.2 Flask Arayüzü

Arayüz Flask ile yazıldı ve formlar için WTForms kullanıldı. Bir filtre formu sınıfında en düşük ve en yüksek değer için metin alanları vardı. Girişin zorunlu olduğu bir doğrulayıcıyla (validator) belirtildi. Siteler için bir seçim alanı ve onay kutuları (checkbox) tanımlandı. JSON verisi Türkçe karakterler düzgün görünsün diye UTF-8 olarak okundu.

GittiGidiyor fiyatlarında virgülden sonra kuruş da geliyordu. Bu yüzden fiyat aralığı karşılaştırması bozuluyordu: kuruşlu gösterim, örneğin 589.900 gibi büyük bir tam sayı olarak okunabiliyordu. Fiyat virgülden ayrılıp yalnız sol tarafı alındı. Hoca kodu grubun kendisinin yazdığına ikna olduğunu söyledi.

26. Restoran Yorumlarında Duygu Analizi

Yirmi beşinci sunum tek kişilikti. Proje bir siteden restoran verisi çekip restoranları sıralıyordu. Kullanıcı bir restorana yorum ekleyebiliyordu. Projenin asıl kısmı yorumun olumlu ya da olumsuz olduğuna karar verip restoranın puanına eklemekti. Restoranın sıralaması buna göre değişiyordu.

Eğitim verisi olarak Amazon, IMDB ve Yelp yorumlarından oluşan hazır bir veri seti kullanıldı. Yorumlar 0-1 etiketliydi. Sınıflandırıcılar Bernoulli Naive Bayes ve destek vektör makinesiydi.

Demoda “good service” yorumu olumlu sınıflandırıldı. SVM’nin doğruluk değeri ekranda yanlış yazılıyordu. Öğrenci, önceki sürümde yaklaşık %79 gösterdiğini söyledi.

Hocanın soruları üzerine akış şöyle netleşti:

- **Metnin vektöre dönüşmesi:** Ön işleme adımı veriyi alıp bozuk satırları kontrol ediyor ve satırları ayırıyordu. Bir vektörleştirici nesnesi (count vectorizer) metni kelimelere ayırıp `fit_transform` ile vektöre çeviriyordu.
- **Etiketin kaynağı:** Vektörün etiketi veri setindeki 0-1 değerleriydi. Metinler ve sonuçlar ayrılıp modele veriliyordu.
- **Başarı ölçümü:** Veri rastgele bölünüyordu (o anda %60 eğitim). Bir karışıklık matrisi gerçek sonuçlarla tahminleri karşılaştırıyor, doğruluk da doğru tahminler bölü bütün tahminler olarak hesaplanıyordu. Bölme rastgele olduğu için doğruluk her çalıştırmada küsuratta değişiyordu.

Hoca önce, yorum girildikten sonra neden ayrıca doğruluk gösterildiğini anlamadı. Öğrenci, sistemin hem yeni yorumu tahmin ettiğini hem de bütün test verisi üzerindeki doğruluğu parantez içinde gösterdiğini açıkladı. Öğrenci kodu yazarken bir makalenin kodundan yardım aldığını ve SVM’yi kendisinin eklediğini söyledi. Hoca açık söylediği için teşekkür etti. Ama sunumda bunun en baştan belirtilmesi gerektiğini, yoksa ilk akla gelen sorunun “acaba ne yaptı” olacağını söyledi.

27. Emoji Tabanlı Duygu Kümeleme

Son sunumda bir öğrenci, Twitter verisinden emoji analizi yapan projeyi sundu. Amaç hangi emojilerin duygusal olarak birbiriyle ilişkili olduğunu, insanların olumlu ve olumsuz duyguları hangi emojilerle gösterdiğini ve genel ruh hâlinin nasıl dağıldığını bulmaktı.

27.1 Veri Setinin Oluşturulması

Birinci adımda Twitter’ın akış (streaming) API’si, Java ile kullanılabilen jar kütüphaneleri üzerinden kullanıldı. Akış sınıfları takip edilen tweetleri emojilere göre ayrılmış dosyalara kaydediyordu. Elde milyonlarca tweet oluştu. Botların attığı aynı tweetler gibi çok sayıda tekrar vardı. Aynı tweetler tek tweet sayıldı. Öğrencinin verdiği bilgiye göre bir milyon tweetten yaklaşık 466 bin benzersiz tweet kaldı.

27.2 Emoji Bağları ve Etiketleme

İkinci adım şöyleydi:

1. Dosya adlarından emojiler bir kümeye alındı. Grubun verdiği bilgiye göre yaklaşık 140 emoji vardı.
2. Her emoji çifti arasındaki bağ sıfırla başlatıldı.
3. Veri seti taranıp aynı tweette birlikte geçen emojilerin bağı birer artırıldı. Örneğin gülen yüz ile kalp çok sık birlikte atıldıysa bağları güçleniyordu.
4. Bağlar sayı olarak geldiği için 0-1 aralığına normalize edildi.
5. %5’ten zayıf bağlar istisna (outlier) kabul edilip silindi. Örneğin gülen yüz ile üzgün yüzün nadiren birlikte atılması.
6. Sekiz etiket belirlendi: Happy, Sad, Angry, Surprised, Sports, Animal, Party ve Nature. Hayvan ve doğa emojileri diğer duygularla çok birlikte atıldığı için bunlar ayrı etiket yapıldı.
7. Her etiket için bir başlangıç emojisi verildi. Diğer emojiler, bağ gücüne göre sıralanıp en yakın 5 emojinin çoğunluk etiketini alarak sırayla etiketlendi ($k = 5$).

27.3 Hocanın k-NN Açıklaması

Hoca k-NN’in de bir sınıflandırıcı olduğunu ve etiketli veri istediğini hatırlattı. Öğrencinin anlattığını k-NN çerçevesine oturtmasını istedi ve yöntemi yeniden anlattı:

- Veri setindeki her veri noktasının bir özellik (feature) vektörü ve bir etiketi vardır.
- Sınıflandırılmak istenen yeni bir noktanın da özellik vektörü vardır, ama etiketi bilinmiyor (soru işaretidir).

- Önce k seçilir, örneğin $k = 5$.
- Yeni noktanın özellik vektörü ile veri setindeki bütün noktaların özellik vektörleri arasındaki uzaklıklar hesaplanır.
- En yakın k nokta bulunur. Bunların arasında baskın (dominant) olan etiket yeni noktanın etiketi olarak atanır.

Hoca "happy ile sad arasındaki uzaklığı nasıl buluyorsun, bir veri noktasının özellik kümesi ve etiketi nedir?" diye sordu. Öğrenci veri noktalarının tweet değil emoji olduğunu, uzaklığın birlikte kullanım sıklığından geldiğini söyledi. Sonunda, başta bir özellik kümesi tanımlamadığını ve k-NN'i baştan kendisi yazarken temel noktayı tam anlamadan kurguladığını kabul etti.

27.4 Sonuçlar

140 emoji 8 etikete kümelendi. Bazı emojiler mantıksız kümelere düşse de çoğunlukla sınıflar doğru çıktı. Parti, hayvan, spor ve özellikle Angry kümeleri iyi sonuç verdi. Sonra bütün tweetler yeniden taranıp her tweet içerdiği emojilere göre kümelere dağıtıldı. Sonuçlar şöyleydi:

- Uç duygular (Happy ve Angry) daha fazla çıktı.
- Genel emoji istatistikleri en çok kullanılan emojilerin olumlu olduğunu gösterse de, Twitter'da Angry, Happy'den bile yüksek çıktı. Öğrenci bunu Twitter'ın daha eleştirel bir platform olmasına bağladı.
- Angry ile Sad'in toplamı olumlu duyguların yaklaşık 1,5 katıydı.
- Olumlu ve olumsuz kategorilerden emojilerin birlikte kullanıldığı "mixed" tweetler yaklaşık 30 bindi. Öğrenci bunu yaklaşık %5 olarak verdi.

  Ders sonrası not: 466 bin tweet içinde 30 bin tweet $30000/466000 \approx \%6{,}4$ eder. Öğrencinin söylediği "yaklaşık %5" değerinden biraz yüksektir. Ayrıca anlatılan yöntem (başlangıç emojilerinden başlayıp birlikte geçme ağı üzerinde komşuların çoğunluk etiketini yaymak), önceden etiketlenmiş bir eğitim kümesi olmadığı için klasik k-NN sınıflandırması değildir. Büyük olasılıkla etiket yayılımına (label propagation) ya da graf tabanlı bir kümelemeye daha yakındır.

Örnek Sorular

Bu derste kare ya da ekran sorusu yok. Aşağıdaki sorular hocanın sunumlar sırasında gruplara sözlü olarak sorduğu teknik sorulardır.

Soru 1 (derste sözlü, 00:59:46): Bir film öneri sistemi şöyle çalışıyor: Sistemin kullanıcısına en benzer 50 kullanıcı Pearson korelasyonuyla bulunuyor, her film için bu kullanıcıların puanlarının benzerlikle ağırlıklı ortalaması alınıyor. Bu yöntem öğe tabanlı işbirlikçi filtreleme midir, içerik tabanlı mıdır?

> **Çözüm (derste verilmedi):** İkisi de değildir. Benzerlik kullanıcılar arasında hesaplandığı için yöntem kullanıcı tabanlı işbirlikçi filtrelemedir (user-based collaborative filtering). Öğe tabanlı yöntemde benzerlik filmler arasında hesaplanır. İçerik tabanlı yöntemde ise kullanıcı puanları yerine filmlerin kendi özellikleri (tür, oyuncu, etiket) kullanılır. Örnekteki iki kullanıcı için skor $(4 \cdot 0{,}7 + 3{,}5 \cdot 0{,}9)/(0{,}7 + 0{,}9) = 5{,}95/1{,}6 \approx 3{,}72$ olur.

Soru 2 (derste sözlü, 01:29:27): Video öneri sisteminde öneri için hangi algoritmayı kullandınız?

Grubun cevabı: Videoların başlıkları ve açıklama metinleri üzerinden bir TF-IDF matrisi oluşturuldu, benzerlik matrisi hesaplanıp benzerlikler sıralandı. Hoca bunu şöyle özetledi: Etiket ve başlıktan bir vektör oluşturulur, bu vektöre en yakın video vektörleri bulunup önerilir. Kullanıcı bilgisi kullanılmadığı için bu, öğe benzerliğine dayanan bir öneridir.

Soru 3 (derste sözlü, 01:52:35 ve 01:53:06): K-means'te K'yı nasıl seçtiniz? En uygun K'yı otomatik seçen kütüphane bunu arka planda nasıl yapıyor?

Grup kütüphanenin ne yaptığını bildiğini, nasıl yaptığını bilmediğini söyledi. Hocanın yorumu: Yalnız ne yaptığını değil, nasıl yaptığını da öğrenmek gerekir.

Çözüm (derste verilmedi): Grubun kullandığı kütüphane altyazıda geçmiyor. K'yı otomatik seçmek için yaygın yöntemler büyük olasılıkla şunlardır. Dirsek (elbow) yöntemi: Her K için küme içi kareler toplamı (inertia) çizilir ve azalmanın belirgin biçimde yavaşladığı K seçilir. Siluet (silhouette) skoru: Her nokta için kendi kümesine ortalama uzaklık a ve en yakın başka kümeye ortalama uzaklık b ise $s = (b - a)/\max(a, b)$ olur ve ortalama s'yi en büyük yapan K seçilir. Grubun 1-9 arası K için çizdiği grafik büyük olasılıkla dirsek grafiğiydi.

Soru 4 (derste sözlü, 02:35:05 ve 02:35:26): Kitap aramasında kitap adının birebir aynı mı yazılması gerekiyor? Sorguyu ve kitap adlarını vektöre çevirip aralarındaki kosinüs benzerliğini hesaplamadınız mı?

Grubun cevabı: Birebir eşleşme gerekiyor, çünkü Firestore'da `LIKE` gibi bir sorgu yok. Arama doğrudan veritabanında yapılıyor.

Çözüm (derste verilmedi): Hocanın önerdiği yaklaşımda sorgu ve her kitap adı aynı sözlük üzerinden (örneğin TF-IDF ile) vektöre çevrilir. Benzerlik $\cos(\vec{q}, \vec{d}) = \dfrac{\vec{q} \cdot \vec{d}}{\|\vec{q}\| \, \|\vec{d}\|}$ ile hesaplanır ve kitaplar bu değere göre sıralanır. Böylece kısaltılmış ya da eksik yazılmış sorgular da ortak kelimeler üzerinden ilgili kitapları bulur. Birebir eşleşme şartı ortadan kalkar.

Soru 5 (derste sözlü, 03:03:23 ve 03:05:20): Log analizinde yalnız istatistikler mi çıkarıldı? Birliktelik algoritmaları ya da sıralı örüntü madenciliği algoritmaları kullanıldı mı? Birliktelik kuralı madenciliği için hangi algoritma kullanıldı?

Öğrencinin cevabı: Apriori algoritması bir Python kütüphanesinden alınıp kullanıldı. Oturumlar IP adresine göre ayrılıp her oturumdaki dokümanlar bir işlem yapıldı. Yaklaşık 45 bin işlemde en düşük destek 0,01 ile 0,001 arasında seçilmeliydi. Hocanın yorumu: Birden fazla birliktelik kuralı algoritmasını karşılaştırmak daha iyi olurdu. Yaklaşık 45 bin işlemde destek 0,01 en az 450, destek 0,001 en az 45 oturumda birlikte geçme demektir.

Soru 6 (derste sözlü, 03:18:45): SVM modeliniz örnek tabanlı (instance-based) bir öğrenme mi? Her yeni veriyle model güncelleniyor mu, yoksa bütün veri setinden tek seferde mi öğreniliyor?

Grubun cevabı: Yeni veri veri setinin sonuna ekleniyor. Yeniden eğitimde bütün veri seti baştan, sıfırdan modele veriliyor. Yani her yeni örnekte artımlı güncelleme yok, toplu (batch) yeniden eğitim var.

Soru 7 (derste sözlü, 03:19:40 – 03:21:27): Kelimelerden oluşan bir cümleyi hangi yöntemle özellik vektörüne çevirip SVM'ye veriyorsunuz?

Grubun cevabı: ML.NET bunu arka planda otomatik yapıyor. Hoca bir örnek yöntem verdi: Word2Vec ile her kelime bir vektöre çevrilir, cümledeki kelime vektörlerinin ortalaması alınır ve bu vektör SVM'ye verilir. Hocaya göre kütüphanenin hangi vektörleştirme tekniğini kullandığını öğrenmek gerekirdi.

Soru 8 (derste sözlü, 03:33:37): Farklı sitelerden gelen ürünleri, örneğin fiyatına göre filtreleyebiliyor musunuz?

Grubun cevabı: Bütün veri CSV'ye yazılıyor, arayüzde filtreleme yok. Hocanın yorumu: Kullanıcı bütün ürünleri değil, örneğin yalnız Nike ürünlerini ya da 43 numara ayakkabıları görmek ister. Birden fazla siteden gelen veri kategori ve fiyat bazında sıralanıp süzülebilmeli.

Soru 9 (derste sözlü, 03:42:15 ve 03:43:24): Yaptığınız önerilerin ne kadar başarılı olduğunu değerlendirdiniz mi?

Öğrenci değerlendirme yapmadığını söyledi. Hocanın yorumu: Yalnız seçilen öğe ile diğer öğeler arasındaki benzerlik gösteriliyor. Önerilerin iyi mi kötü mü olduğunu bilmek için ayrı bir değerlendirme yöntemi gerekir.

Çözüm (derste verilmedi): Kullanıcıların puanlarının bir kısmı test için ayrılır. Model bu puanları tahmin eder ve hata büyük olasılıkla RMSE $= \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{r}_i - r_i)^2}$ ya da MAE ile ölçülür. Liste biçimindeki önerilerde ilk k öneri içinde kullanıcının

test kümesinde gerçekten beğendiği öğelerin oranı (precision@k) ve bu öğelerin ne kadarının listeye girdiği (recall@k) ölçülür.

Soru 10 (derste sözlü, 03:50:59 ve 03:51:46): Tweetlerin olumlu, olumsuz ya da nötr olduğunu neye göre belirliyorsunuz? Hiçbir yerde hazır bir veri seti yok muydu?

Grubun cevabı: TextBlob kütüphanesinin duygu fonksiyonu kullanıldı, model eğitilmedi, veri seti bulunamadı. Hocanın cevabı: Cümleler Word2Vec gibi bir yöntemle vektöre çevrilmeli, etiketli veri hâline getirilmeli ve en az üç farklı makine öğrenmesi algoritması eğitilmeliydi. Az veriyle de yapılabilir: Bir grup 500 etiketli veriyi elle oluşturdu.

Soru 11 (derste sözlü, 04:01:37 – 04:01:45): URL'den çekilen metinden anlamlı anahtar kelimeleri nasıl çıkarıyorsun? Yan yana gelmiş karakterlerin anlamlı bir kelime olduğuna nasıl karar veriyorsun?

Öğrencinin cevabı: NLTK durak kelimelerini kullanan hazır RAKE kütüphanesiyle. Öğrenci bu kısmı modellemediğini kabul etti. Hocanın yorumu: Projenin en önemli kısmı bir kütüphaneye yaptırılmış. Derste işlenen tekniklerin kodun içinde görülmesi gerekir.

Soru 12 (derste sözlü, 04:21:33 – 04:24:18): Yorumları neye çeviriyorsun? Vektörün etiketi nereden geliyor? Bir sınıflandırıcının performansını normalde nasıl ölçeriz, sen bunu yaptın mı?

Öğrencinin cevabı: Yorumlar count vectorizer ile vektöre çevriliyor. Etiketler veri setindeki 0-1 değerlerinden geliyor. Veri rastgele eğitim ve test olarak bölünüyor, test verisi üzerinde karışıklık matrisi oluşturulup doğruluk hesaplanıyor. Hocanın tanımı: Veri eğitim ve test olarak ayrılır, eğitim verisinden öğrenilir, test verisi üzerinde denenip sonuç alınır.

Soru 13 (derste sözlü, 04:42:36 ve 04:44:47): k-NN ile emoji sınıflandırırken happy ile sad arasındaki uzaklığı nasıl buluyorsun? Bir veri noktasının özellik kümesi nedir, etiketi nedir?

Hoca k-NN'i şöyle anlattı: Her veri noktasının bir özellik vektörü ve etiketi vardır. Etiketi bilinmeyen yeni noktanın özellik vektörü ile bütün noktalar arasındaki uzaklıklar hesaplanır, en yakın k nokta bulunur ve bunlar arasında baskın olan etiket atanır. Öğrenci başta bir özellik kümesi tanımlamadığını kabul etti.

> **Çözüm (derste verilmedi):** Bu projede bir emojinin özellik vektörü, büyük olasılıkla o emojinin diğer 139 emojiyle normalize edilmiş birlikte geçme değerlerinden oluşan vektör olarak tanımlanabilirdi. İki emoji arasındaki uzaklık bu vektörler arasındaki kosinüs ya da Öklid uzaklığı olurdu. Etiketli eğitim kümesi, elle etiketlenmiş emojilerden (örneğin her kategoriden birkaç başlangıç emojisi) oluşurdu. Kalan emojiler bu kümedeki en yakın k komşunun çoğunluk etiketiyle sınıflandırılırdı.

Dersin Toparlaması

Bu ders, dönem boyunca işlenen konuların öğrenci projelerinde nasıl uygulandığını gösterdi. Web tarayıcı projelerinde Scrapy, Selenium, Puppeteer ve `requests` ile statik ve dinamik sayfalardan veri toplandı. Veri CSV, JSON, MongoDB, Firebase ya da Firestore'da saklandı ve Django, Flask, React ya da Next.js arayüzleriyle gösterildi. Sınıflandırma projelerinde kelime torbası, TF-IDF, frekans özellikleri ve Word2Vec gibi temsillerle Naive Bayes, Logistic Regression, Decision Tree, Random Forest, SVM, k-NN ve XGBoost karşılaştırıldı. Öneri projelerinde TF-IDF ile kosinüs benzerliği, Pearson korelasyonu ve k-NN ile kullanıcı ve öğe tabanlı işbirlikçi filtreleme kullanıldı. Duygu analizi projelerinde tweetler, yorumlar ve gazete haberleri incelendi.

Hocanın sorularında dört ölçüt öne çıktı. Birincisi, tarayıcı projelerinde birden fazla siteden gelen verinin kategori ve fiyata göre süzülüp sıralanabilmesi. İkincisi, sınıflandırma projelerinde kendi etiketli verinizle en az üç algoritmanın eğitilip precision, recall, F1 ve karışıklık matrisiyle karşılaştırılması. Üçüncüsü, öneri sistemlerinde önerilerin bir değerlendirme yöntemiyle ölçülmesi. Dördüncüsü, kullanılan kütüphanelerin yalnız ne yaptığının değil, nasıl yaptığının da bilinmesi ve derste işlenen tekniklerin kodun içinde görülmesi. Sunumların ardından final sınavı haftasına geçildi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.